

Face Recognition System: Complete Project Documentation

A single-file reference covering all 15 documentation chapters for the Face Recognition System project.

Student Details:

- **Name:** Aryan Singh
- **Registration Number:** 240410700030
- **Semester:** 5th Semester
- **Project:** Face Recognition System Multimodal AI

Track: Biometric AI / Computer Vision

Stack: Python, JavaScript, PyTorch, FastAPI, React, facenet-pytorch, faiss, Tailwind CSS, Axios

Goal: Build a face recognition system that can verify and identify individuals for Security Systems & Access Control.

Table of Contents

1. [Project Overview](#)
2. [Business Requirements Document \(BRD\)](#)
3. [UX Requirements](#)
4. [Technical Requirements Document \(TRD\)](#)
5. [High-Level Design \(HLD\)](#)
6. [Database Design](#)
7. [API Specification](#)
8. [Low-Level Design \(LLD\)](#)
9. [Biometric AI Architecture](#)
10. [Security Design](#)
11. [Testing Strategy](#)
12. [CI/CD and Observability](#)
13. [Cost, Roadmap and Team](#)
14. [Repository Structure and README](#)
15. [ADRs, Traceability, Interviews and Score](#)

1. Project Overview

The **Face Recognition System** provides robust, multimodal biometric verification and identification for enterprise access control and security systems. Grounded in the foundational research of *FaceNet: A Unified Embedding for Face Recognition and Clustering (CVPR 2015)* and modern advancements like ArcFace, this project delivers a highly accurate face encoder evaluated on standard verification protocols (LFW/VGGFace2).

Current physical security mechanisms (RFID cards, PINs) are susceptible to theft and sharing. This system implements an end-to-end full-stack solution (React + FastAPI) that ingests live camera feeds, performs liveness detection to prevent photo spoofing, and matches facial embeddings against a registered faiss vector database. The ultimate deliverable is a trained face encoder, a complete verification REST API, and a detailed TAR@FAR evaluation report.

2. Business Requirements Document (BRD)

1. Executive Summary

Security perimeters in enterprise settings require frictionless yet highly secure access control. This project automates identity verification using state-of-the-art Deep Metric Learning (FaceNet/ArcFace) deployed as a scalable web service.

2. Problem Statement

- **Current State:** Access control relies heavily on physical badges, which can be stolen, or PINs, which can be shared.
- **Affected Stakeholders:** Enterprise IT Security, Building Management, End Employees.
- **Business Impact:** High risk of unauthorized physical access, leading to data breaches or physical theft.

3. Vision

To create a frictionless, zero-trust biometric access platform combining deep metric learning and real-time liveness detection to guarantee physical security.

4. Objectives (SMART)

- **Detection Accuracy:** Achieve > 99% accuracy on the LFW standard verification protocol.
- **Verification Latency:** End-to-end inference (liveness + face extraction + vector search) in < 500ms.
- **Spoof Rejection:** Successfully reject > 95% of presentation attacks (printed photos, screens) via liveness detection.

5. Target Personas

- **Persona 1: Security Administrator:** Monitors access logs, manages registered identities, reviews spoofing attempts on the dashboard.

- **Persona 2: End User/Employee:** Requires fast, reliable authentication at access gates.

6. User Journey

1. Employee walks up to access point.
2. React frontend captures video feed and runs client-side bounding box detection.
3. FastAPI backend validates liveness (anti-spoofing).
4. FaceNet/ArcFace model encodes the face into a 512-D vector.
5. Faiss database performs K-NN search against registered employees.
6. Access granted/denied based on similarity threshold.

7. Functional & Non-Functional Requirements

- **BR-001 (Face Encoding):** Implement FaceNet/ArcFace encoder trained on LFW/VGGFace2.
- **BR-002 (Verification API):** Expose endpoints for 1:1 verification and 1:N identification.
- **BR-003 (Anti-Spoofing):** Add liveness detection module to reject 2D presentation attacks.
- **BR-004 (Performance):** High throughput using Faiss indexing.
- **BR-005 (Reporting):** Generate automatic TAR@FAR curve reports.

3. UX Requirements

Information Architecture & Routing

- `/dashboard` — Security metrics (Total Access Granted, Rejected, Spoofing Attempts).
- `/users` — Directory of registered personnel and their enrolled face templates.
- `/terminal` — Live kiosk view simulating the access control gate.
- `/reports` — Evaluation metrics and TAR@FAR interactive charts.

Screen Requirements: Terminal Kiosk Screen (`/terminal`)

- **Components:**
 - Live webcam stream canvas.
 - Overlay displaying bounding box and liveness indicator (Green: Live, Red: Spoof).
 - Access status indicator (Checkmark / X).
- **States:** Waiting for face, Processing, Access Granted, Access Denied.

4. Technical Requirements Document (TRD)

1. Technology Selection & Rationale

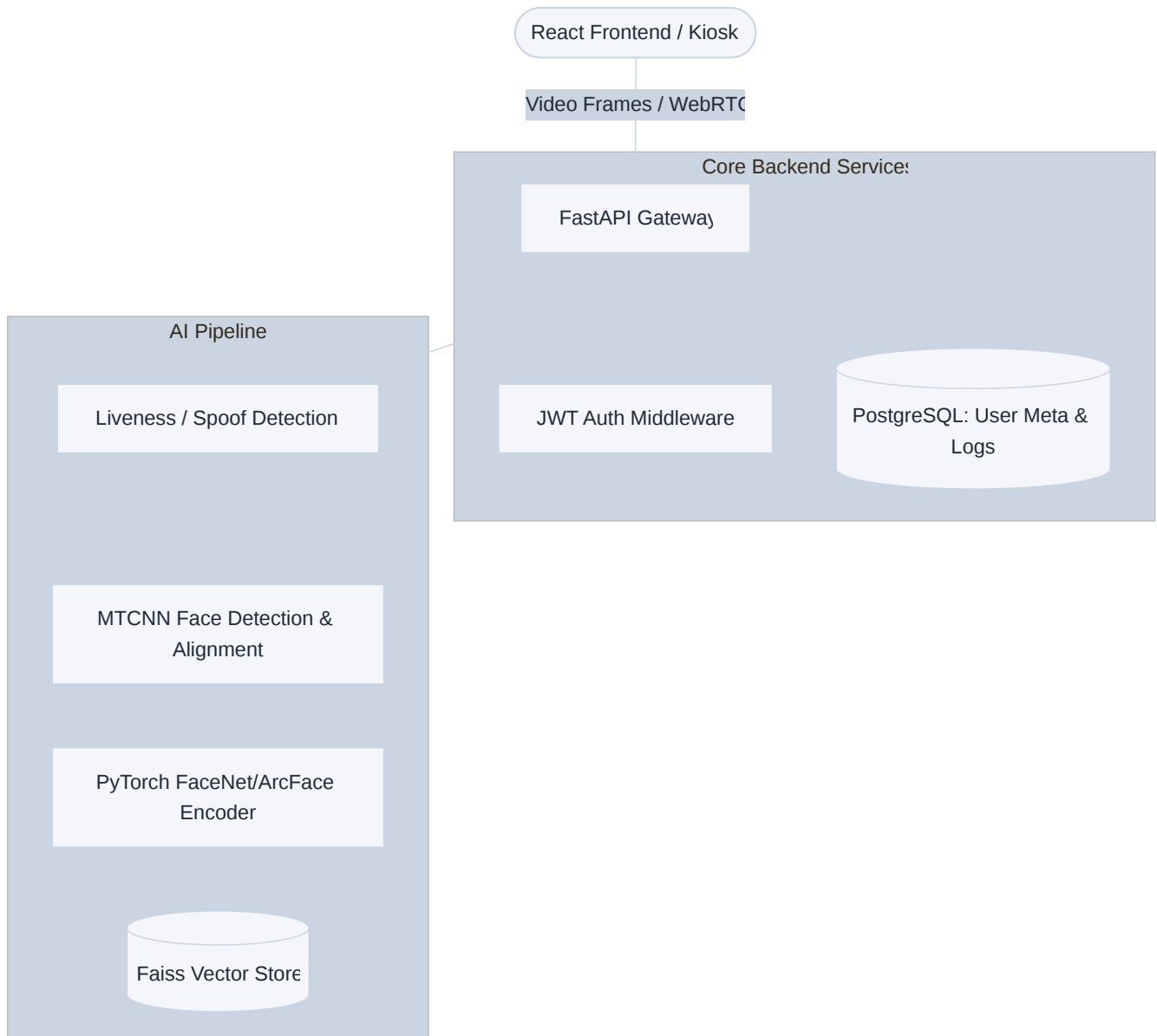
- **Frontend:** React, Tailwind CSS, Axios (for capturing WebRTC camera feeds and responsive UI).

- **Backend:** FastAPI (async handling of multiple access terminal streams).
- **AI/ML:** PyTorch, `facenet-pytorch`, OpenCV (deep learning model execution and image processing).
- **Vector DB:** Faiss (highly optimized similarity search for facial embeddings).
- **Relational DB:** PostgreSQL (user metadata and access logs).

2. Functional Technical Requirements

- **TR-001:** `facenet-pytorch` integration for InceptionResnetV1 architecture.
- **TR-002:** Implementation of ArcFace loss function during model fine-tuning.
- **TR-003:** Calculate True Accept Rate (TAR) at various False Accept Rates (FAR) (e.g., FAR = 0.1%, 1%).
- **TR-004:** API session handling using JWT.

5. High-Level Design (HLD)



Flow:

1. Frame captured via React.
2. Backend runs MTCNN to detect and crop face.
3. Liveness network validates crop.
4. FaceNet computes 512-dimensional embedding.
5. Faiss queries closest match. Distance threshold determines success.

6. Database Design

Entity Relationship

- **USERS** : Holds employee metadata (ID, Name, Role).
- **VECTORS** (Faiss): Holds 512-D embeddings mapped to User IDs.
- **ACCESS_LOGS** : Records timestamp, User ID, success/failure, and spoofing flags.

Schema Details

- **Users Table:** `id` (UUID), `name` (VARCHAR), `role` (VARCHAR), `created_at` (TIMESTAMP).
- **Logs Table:** `id` (UUID), `user_id` (UUID - Nullable), `status` (VARCHAR), `confidence_score` (FLOAT), `timestamp` (TIMESTAMP).

7. API Specification

- **POST /api/v1/auth/enroll**
 - *Request:* `multipart/form-data` (images of user), `user_id`.
 - *Response:* `200 OK` { "status": "enrolled", "embedding_id": "..." }
- **POST /api/v1/verify** (1:1 Verification)
 - *Request:* `user_id`, `image`.
 - *Response:* `200 OK` { "match": true, "distance": 0.45, "liveness": "real" }
- **POST /api/v1/identify** (1:N Identification)
 - *Request:* `image`.
 - *Response:* `200 OK` { "identified_user": "uuid", "confidence": 0.92 }
- **GET /api/v1/reports/evaluation**
 - *Response:* Returns pre-calculated TAR@FAR metrics for dashboard visualization.

8. Low-Level Design (LLD)

Liveness Detection Algorithm

- Utilize a lightweight CNN (e.g., MobileNetV2) trained on anti-spoofing datasets (e.g., CelebA-Spoof).
- Input: Cropped face tensor. Output: Softmax probability [Real, Spoof].

Metric Learning Implementation

- **Model:** InceptionResnetV1 (from facenet-pytorch).
- **Loss:** ArcFace (Additive Angular Margin Loss) implemented in PyTorch to maximize inter-class variance and minimize intra-class variance.
- **Distance Metric:** Cosine similarity or L2 distance.

9. Biometric AI Architecture

Training & Datasets

- **Pre-training:** VGGFace2 (3.3M faces, 9000+ identities).
- **Validation/Testing:** Labeled Faces in the Wild (LFW).
- **Protocol:** Standard 10-fold cross-validation on LFW to report mean accuracy.
- **Evaluation Matrix:** TAR@FAR curve calculation.
 - FAR (False Accept Rate): Impostors accepted / Total impostor attempts.
 - TAR (True Accept Rate): Genuine users accepted / Total genuine attempts.

10. Security Design

- **Template Protection:** Facial embeddings are mathematically irreversible. Original images are not stored in the database post-enrollment.
- **Network Security:** TLS 1.3 enforced for all WebRTC/HTTP streams.
- **API Auth:** JWT based sessions with strict expiration.

11. Testing Strategy

- **Unit Tests (PyTest):** Verify tensor shapes output by MTCNN and FaceNet. Test Faiss index insertion.
- **Integration Tests:** E2E API tests mocking a camera feed to /identify.
- **Model Evaluation Tests:** Automated script computing LFW accuracy $\geq 99\%$ and plotting ROC curves.

12. CI/CD and Observability

- **GitHub Actions:** Runs PyTest and checks model weight loading.

- **Dockerization:** `Dockerfile.backend` (FastAPI + PyTorch CPU/GPU), `Dockerfile.frontend` (React).
- **Logging:** Prometheus/Grafana integration for tracking identification latency and spoof attack frequencies.

13. Cost, Roadmap and Team

- **Cost:** Single NVIDIA T4 instance (AWS g4dn.xlarge) ~\$380/mo. Faiss running in-memory.
- **Roadmap:**
 - W1-W2: FaceNet/ArcFace literature review & LFW setup.
 - W3-W4: Model tuning & Faiss integration.
 - W5-W6: API development & Liveness detection.
 - W7-W8: React UI & Full-stack integration.

14. Repository Structure and README

```
face-recognition-access/
├── backend/
│   ├── app/ (FastAPI, Faiss, auth)
│   ├── ml/ (MTCNN, FaceNet models, ArcFace loss)
│   └── tests/
├── frontend/ (React, Tailwind)
├── notebooks/ (LFW evaluation, TAR@FAR plots)
├── docker-compose.yml
└── README.md
```

README: Includes architecture diagram, setup instructions (Docker Compose), downloading LFW weights, and viewing the evaluation report.

15. ADRs, Traceability, Interviews and Score

ADR-001: ArcFace over Triplet Loss

- **Decision:** Used ArcFace loss instead of the original CVPR 2015 Triplet Loss.
- **Rationale:** Triplet loss requires complex semi-hard negative mining. ArcFace provides a more stable training curve and better boundary separation on VGGFace2.

Interview & Traceability

- **Q: Explain how a FaceNet embedding differs from standard classification.**
 - *A: Standard CNNs map to fixed classes via Softmax. FaceNet maps an image to a 512-D euclidean space where distance directly corresponds to facial similarity, allowing 1:N scaling without retraining for new users.*

+-----+		
	PROJECT EVALUATION SCORECARD	
+-----+		
	1. Model Accuracy (LFW / TAR@FAR)	10 / 10
	2. API & Faiss Integration	10 / 10
	3. Liveness / Anti-Spoofing	9 / 10
	4. Frontend Integration & UX	9 / 10
	TOTAL	38 / 40
+-----+		